

Database Concurrency & Locking Strategies

Engineering Guide for High-Transaction Order Systems

1. Overview of the Race Condition

In distributed environments like DropBox Nigeria, multiple app workers may attempt to read and update the same inventory record simultaneously. Standard transaction isolation often fails to prevent "Over-selling" because two processes may read identical state before either commits an update.

2. Comparison of Locking Strategies

Strategy	Mechanism	Pros	Cons
Pessimistic Locking	Uses <code>SELECT FOR UPDATE</code> to lock the row as soon as it is read.	Total data integrity; prevents any concurrent access.	High risk of deadlocks; limits throughput.
Atomic Updates	Combines check and action: <code>UPDATE ... WHERE stock > 0.</code>	Fastest execution; handles concurrency at the DB engine level.	Limited to simple decrement/increment logic.
Optimistic Locking	Uses a <code>version</code> column. Fails if version changed since read.	No DB locks; high performance for read-heavy loads.	Requires retry logic in application code.

3. Implementation Guide: Atomic Update

This is the recommended approach for the **DropBox Nigeria** inventory system. It ensures that the decrement only happens if the stock is currently sufficient.

```
-- Atomic Transaction with Result Capture
UPDATE inventory
SET stock = stock - 1
WHERE id = :product_id AND stock > 0
```

```
RETURNING stock;
```

Execution Logic:

- If the query returns a row, the order is successful.
- If the query returns 0 rows, the item is out of stock.

| 4. Scaling with Idempotency

To prevent network retries from creating duplicate orders, use a unique request key (e.g., `user_id + product_id + session_id`) stored in Redis for 60 seconds.